

Software Requirement Specification

Problem Statement: A few sentences to describe the problem you are trying to solve, i.e., justify why this software is needed.

- Hopkins students who just started to invest their money have no idea where to begin. They could just put all their money in treasury bonds or S&P500, but for beginners or those who want to develop strategies to get better returns, stock pick articles and market news can be overwhelming. Moreover, students who start trading their strategies can lose money. This is especially true for beginners because there are so many experienced traders, trading companies, and algorithms out there which they will be up against. This app will allow Hopkins students to practice their trading skills and intuitions in a low stake environment.

Potential Clients: Who are influenced by this problem and would benefit from the proposed solution? (i.e. the potential users)

- College students who just started to invest / want to learn more about investing (starting with Hopkins students)

Proposed Solution: Write a few sentences that describe how a software solution will solve the problem described above.

- The proposed solution is an interactive platform that helps students learn investing through simulated trading, guided strategy testing, and educational support. By combining analytics, AI assistance, and engaging features, the app helps users gradually build confidence and decision-making skills in a structured, low-pressure environment.

Functional Requirements: List the (functional) requirements that software needs to have in order to solve the problem stated above. List these in role-goal-benefit format. It is useful to try to group the requirements into those that are essential (must have), and those which are non-essential (but nice to have).

Must have

- As a new or returning trader, I want to create an account and log in securely so that my simulated trades, progress, and strategies are saved and tied to me across sessions.
- As someone learning how stock markets work, I want to simulate trading using real market data without real money so that I can practice strategies and understand market behavior without financial risk.
- As a user who wants to track improvement, I want to see my total portfolio value based on my simulated trades so that I can monitor performance and see how my decisions affect outcomes over time.
- As a learner with limited time, I want access to short lessons and quick tips so that I can understand key trading concepts without needing extensive prior knowledge.
- As a user with trading ideas but little or no programming experience, I want to describe trading strategies in plain English so that I can test ideas without learning how to code.
- As a user experimenting with strategy-based trading, I want my plain-English strategy to be automatically converted into executable code so that it can be tested rather than remaining theoretical.
- As a user trying to evaluate a trading strategy, I want to run my strategy against real historical market data and receive performance metrics so that I can understand how effective the strategy would have been.
- As a user looking to improve decision-making, I want to receive an assessment of my strategy so that I can identify weaknesses and refine my approach.

Nice to have

- As a more advanced or curious user, I want to simulate high-frequency or algorithmic trading strategies so that I can explore more complex trading techniques in a risk-free environment.
- As a user seeking guidance, I want personalized, AI-powered trade and strategy suggestions based on my activity and market data so that I can make more informed decisions.
- As a user motivated by comparison, I want to see a leaderboard ranking users by performance so that I can benchmark my results against others.
- As a socially engaged user, I want to invite friends and earn rewards so that I am incentivized to stay active and unlock additional features.
- As a user who benefits from structure, I want to participate in weekly or monthly trading challenges so that I stay engaged and build consistent learning habits.
- As a user exploring investment opportunities, I want to swipe through stocks or options with brief insights so that I can quickly discover and shortlist assets.

Non-functional Requirements:

- As a user who frequently tests ideas, I want simulations and strategy tests to run quickly so that I receive immediate feedback and can iterate efficiently.
- As an active user, I want the app to remain responsive even during high traffic so that performance issues do not disrupt my experience.
- As a user who values privacy and trust, I want my personal data, strategies, and activity to be securely protected so that I feel confident using the platform.
- As a user new to trading tools, I want an intuitive and easy-to-navigate interface so that I can use the app effectively without feeling overwhelmed.

Software Architecture & Technology Stack: Will this be a Web/desktop/mobile (all, or some other kind of) application?

Would it conform to specific software architecture? What programming languages, frameworks, databases, ..., will be used to develop and deploy the software?

Web-based (possibly mobile depending on time constraint) client-server system:

Frontend: Responsible for user interactions, account management, simulated trading interfaces, portfolio visualization, etc.

- Javascript React
- Communicate with the backend using RESTful APIs.

Backend: Handles core application logic like user authentication, strategy processing, trade simulation, data management, etc using service-oriented architecture

- Python (Good for data, analytics)
- FastAPI
- AI-based strategy interpretation, translating natural language descriptions into trading logic.

Data:

- Market data extracted from financial data APIs and possible scrapping
- Relational databases like PostgreSQL used to store user accounts, simulated trades and strategy metadata, etc for structured queries.
- Encryption for sensitive data

Cloud-Based Infrastructure: Allow for scaling and responsiveness for demand and traffic.

Similar Apps: List a few similar applications to the one you are developing. Don't be eager to conclude no similar app exists! There is always something similar to what you are building! Finding those will help you to better specify your project. ***You must be prepared to explain how your app is different from the existing ones.***

QuantConnect - allows users to define strategies and backtest them against historical market data. has the pipeline (plain english strategy -> python code -> historic data backtest) implemented. Difference: focuses on advanced users and offers a pipeline from strategy definition to code execution. Differs from our educational features targeting beginners.

Investopedia Simulator - allows users to practice trading with virtual money using real market data. However, it focuses mainly on manual trading and lacks structured strategy testing, automated backtesting, or AI-driven feedback.

Kalshi Demo - Gives demo version of its app, but lacks educational features and AI- driven feedback. Only integrates trading/betting into one platform.